Министерство цифрового развития, связи и массовых коммуникаций
Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования «Сибирский государственный университет
телекоммуникаций и информатики»
(СибГУТИ)

Кафедра прикладной математики и кибернетики

Операционные системы

Лабораторная работа №10

Выполнили: Студенты 3-го курса,
группы ИП-111
Гердележов Даниил Дмитриевич
Махаева Алина Витальевна
Клепче Георгий Васильевич

Проверил преподаватель:
Малков Евгений Александрович

Новосибирск, 2023

Оглавление

Задание: 3
Цель: 3
Ход работы: 4
Вывод: 7
Листинг: 8
Pthreads.cpp 8
thread.cpp 9
lab9.c 10
peterson.cpp 11

Задание: протестируйте программы лабораторных 8 и 9, используя программную реализацию алгоритма Петерсона, запуская их на одном, двух и нескольких ядрах. Протестируйте модифицированный на основе атомарных функций код алгоритма Петерсона используя различные модели упорядочения выполнения инструкций кода.

Цель: знакомство с атомарными функциями

Ход работы:

Доработаем две программы лабораторной работы №8, которые вычисляют сумму чисел от 0 до 1 000 000 на основе последовательного кода, интерфейсов Pthreads и C++11.

Мы изменили некоторые аспекты кода, чтобы обеспечить корректную работу на нескольких ядрах процессора. В частности, мы изменили структуру **ThreadArgs** для хранения ссылки на вектор чисел, начального и конечного индексов для определения диапазона, который будет обрабатываться каждым потоком и переменной для хранения результата.

Одним из ключевых изменений было использование динамического выделения памяти для каждого экземпляра **ThreadArgs**, что позволило передать уникальный диапазон чисел в каждый поток. Кроме того, мы использовали функцию **pthread_join**, чтобы дождаться завершения всех потоков перед подсчетом окончательной суммы.

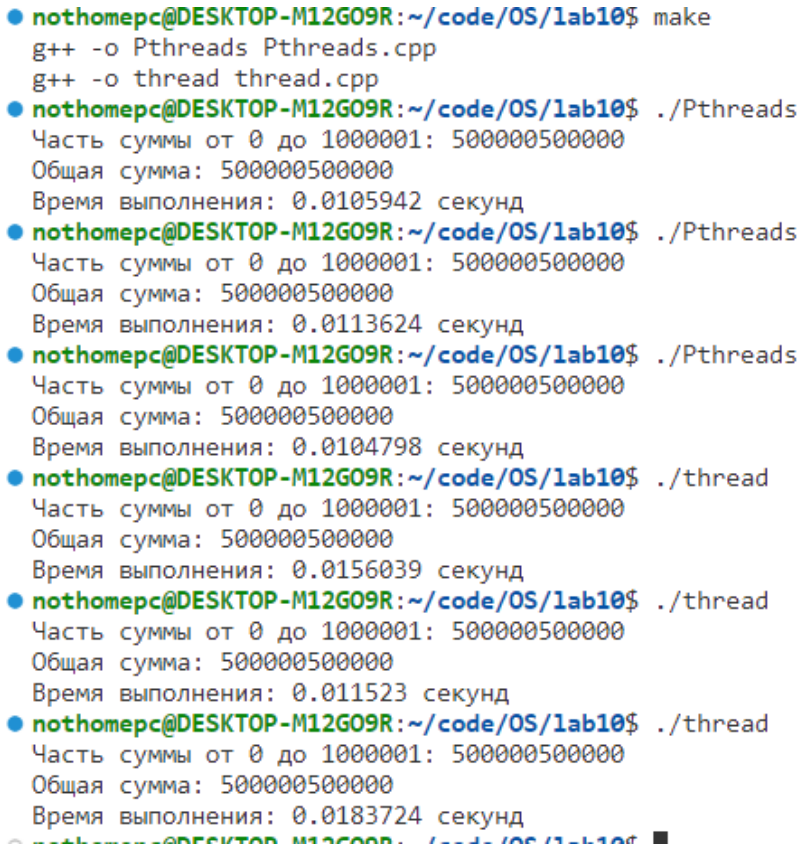


Рис. 1 – Компиляция и несколько результатов работы обеих программ на одном ядре.

```
nothomepc@DESKTOP-M12GO9R:~/code/OS/lab10$ make
g++ -o Pthreads Pthreads.cpp
g++ -o thread thread.cpp
nothomepc@DESKTOP-M12GO9R:~/code/OS/lab10$ ./Pthreads
Часть суммы от 0 до 500000: 124999750000
Часть суммы от 500000 до 1000001: 375000750000
Общая сумма: 500000500000
Время выполнения: 0.00589103 секунд
nothomepc@DESKTOP-M12GO9R:~/code/OS/lab10$ ./Pthreads
Часть суммы от 0 до 500000: 124999750000
Часть суммы от 500000 до 1000001: 375000750000
Общая сумма: 500000500000
Время выполнения: 0.00742842 секунд
nothomepc@DESKTOP-M12GO9R:~/code/OS/lab10$ ./Pthreads
Часть суммы от 0 до 500000: 124999750000
Часть суммы от 500000 до 1000001: 375000750000
Общая сумма: 500000500000
Время выполнения: 0.00829398 секунд
nothomepc@DESKTOP-M12GO9R:~/code/OS/lab10$ ./thread
Часть суммы от 0 до 500000: 124999750000
Часть суммы от 500000 до 1000001: 375000750000
Общая сумма: 500000500000
Время выполнения: 0.0125593 секунд
nothomepc@DESKTOP-M12GO9R:~/code/OS/lab10$ ./thread
Часть суммы от 0 до 500000: 124999750000
Часть суммы от 500000 до 1000001: 375000750000
Общая сумма: 500000500000
Время выполнения: 0.011611 секунд
nothomepc@DESKTOP-M12GO9R:~/code/OS/lab10$ ./thread
Часть суммы от 0 до 500000: 124999750000
Часть суммы от 500000 до 1000001: 375000750000
Общая сумма: 500000500000
Время выполнения: 0.00641788 секунд
nothomepc@DESKTOP-M12GO9R:~/code/OS/lab10$ ./thread
Часть суммы от 500000 до 1000001: 375000750000
Часть суммы от 0 до 500000: 124999750000
Общая сумма: 500000500000
Время выполнения: 0.00656809 секунд
```

Рис. 2 – Компиляция и несколько результатов работы обеих программ на двух ядрах.

Аналогично были сделаны замеры при использовании 4-ех ядер.

Таблица 1 – Средний результат вычисление суммы чисел от 0 до 1 000 000 на разном количестве ядер:

	1		2		4	
	Pthreads	thread	Pthreads	thread	Pthreads	thread
Время выполнения	0,01123680	0,01152340	0,01093190	0,01221600	0,00586535	0,02216060
	0,01773630	0,01235770	0,00534008	0,01055040	0,00461823	0,00595554
	0,01184010	0,01209980	0,00654532	0,01710930	0,00676912	0,00615809
	0,01221850	0,01229020	0,00907060	0,00640478	0,00881350	0,00687585
	0,01194350	0,01150550	0,00666224	0,01448500	0,01624420	0,00504132
	0,01111330	0,01241060	0,00695065	0,00561910	0,00868577	0,00554669
	0,01528190	0,01203560	0,01002910	0,01140920	0,00459636	0,00614183
	0,01122030	0,01190650	0,00649495	0,01430550	0,00506500	0,00855692
	0,01329640	0,01182720	0,00667439	0,00690063	0,01022320	0,01340750
	0,01329640	0,01293170	0,00600225	0,01093190	0,01434700	0,00598997
Ср. знач	0,01291835	0,01208882	0,00747015	0,01099318	0,00852277	0,00858343
	Прирост относительно 1 ядра					
	-	-	42.18%	9.06%	34.03%	29%

Как можно заметить прирост скорости расчётов колеблется от 9% до 42%.

Теперь доработаем программу лабораторной работы №9. Одно ядро будет заполнять в sh “Hello”, второе – “Bye_U” (код программы представлен в листинге).

```
nothomepc@DESKTOP-M12GO9R:~/code/OS/lab10$ gcc -o lab10 -Wall lab9.c
nothomepc@DESKTOP-M12GO9R:~/code/OS/lab10$ ./lab10
Bye_u
Bye_u
Bye_u
Bye_u
Bye_u
Hello
Hello
Hello
Hello
Hello
Bye_u
Bye_u
^C
nothomepc@DESKTOP-M12GO9R:~/code/OS/lab10$
```

Рис. 3 – Компиляция и результат работы модифицированной программы лабораторной работы №9.

Создадим модифицированный на основе атомарных функций алгоритм Петерсона (код программы представлен в листинге). Протестируем программу:

```
nothomepc@DESKTOP-M12GO9R:~/code/OS/lab10$ g++ -o peterson -Wall peterson.cpp
nothomepc@DESKTOP-M12GO9R:~/code/OS/lab10$ ./peterson
Процесс 0 вошел в критическую секцию.
Процесс 0 вышел из критической секции.
Процесс 1 вошел в критическую секцию.
Процесс 1 вышел из критической секции.
nothomepc@DESKTOP-M12GO9R:~/code/OS/lab10$
```

Рис. 4 - Компиляция и результат работы модифицированного на основе атомарных функций алгоритма Петерсона.

Из результата работы программы можно сделать вывод, что оба процесса (0 и 1) успешно вошли в свои критические секции и вышли из них.

Вывод:

- Мы протестировали программы лабораторных 8 и 9, используя программную реализацию алгоритма Петерсона, запускали их на одном, двух и четырёх ядрах. Протестировали модифицированный на основе атомарных функций код алгоритма Петерсона, используя различные модели упорядочения выполнения инструкций кода.
- Познакомились с атомарными функциями.

Листинг:

Pthreads.cpp

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

char sh[6];
int turn = 0;
int interested[2] = {0, 0};

void lock(int self) {
    interested[self] = 1;
    turn = 1 - self;

    while (turn == 1 - self && interested[1 - self] == 1)
        ; // Ожидаем освобождения ресурса
}

void unlock(int self) {
    interested[self] = 0;
}

void* Thread(void* pParams) {
    int counter = 0;
    int id = *((int*)pParams);
    while (1) {
        lock(id);

        if (counter % 2) {
            sh[0] = 'H'; sh[1] = 'e'; sh[2] = 'l'; sh[3] = 'l'; sh[4] = 'o'; sh[5]
= '\0';
        } else {
            sh[0] = 'B'; sh[1] = 'y'; sh[2] = 'e'; sh[3] = '_'; sh[4] = 'u'; sh[5]
= '\0';
        }
        counter++;

        unlock(id);

        usleep(500000);
    }
    return NULL;
}

int main() {
    pthread_t thread_id[2];
    int ids[2] = {0, 1};
    pthread_create(&thread_id[0], NULL, &Thread, &ids[0]);
    pthread_create(&thread_id[1], NULL, &Thread, &ids[1]);
```

```
    while (1) {
        printf("%s\n", sh);
        usleep(100000);
    }

    pthread_join(thread_id[0], NULL);
    pthread_join(thread_id[1], NULL);
    return 0;
}
```

thread.cpp

```
#include <iostream>
#include <vector>
#include <thread>
#include <numeric>
#include <chrono>
#include <mutex>

void sumRange(const std::vector<int>& numbers, size_t start, size_t end, long long
int& result, std::mutex& mtx) {
    long long int partialSum = std::accumulate(numbers.begin() + start,
numbers.begin() + end, 0LL); // Используем 0LL для явного указания на тип long long
int
    mtx.lock();
    std::cout << "Часть суммы от " << start << " до " << end << ": " << partialSum
<< std::endl;
    mtx.unlock();

    result = partialSum;
}

int main() {
    std::vector<int> numbers(1000001);
    std::iota(numbers.begin(), numbers.end(), 0);
    int numThreads = 4;

    std::vector<std::thread> threads(numThreads);
    std::vector<long long int> partialSums(numThreads); // Изменяем тип данных для
частичных сумм
    std::mutex mtx;

    size_t chunkSize = numbers.size() / numThreads;

    auto start = std::chrono::high_resolution_clock::now();

    for (int i = 0; i < numThreads; i++) {
        size_t start = i * chunkSize;
        size_t end = (i == numThreads - 1) ? numbers.size() : (start + chunkSize);
        threads[i] = std::thread(sumRange, std::ref(numbers), start, end,
std::ref(partialSums[i]), std::ref(mtx));
```

```
    }

    for (int i = 0; i < numThreads; i++) {
        threads[i].join();
    }

    long long int totalSum = std::accumulate(partialSums.begin(),
partialSums.end(), 0LL); // Используем 0LL для явного указания на тип long long int

    auto end = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> duration = end - start;

    std::cout << "Общая сумма: " << totalSum << std::endl;
    std::cout << "Время выполнения: " << duration.count() << " секунд" <<
std::endl;

    return 0;
}
```

lab9.c

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

char sh[6];
int turn = 0;
int interested[2] = {0, 0};

void lock(int self) {
    interested[self] = 1;
    turn = 1 - self;

    while (turn == 1 - self && interested[1 - self] == 1)
        ; // Ожидаем освобождения ресурса
}

void unlock(int self) {
    interested[self] = 0;
}

void* Thread(void* pParams) {
    int counter = 0;
    int id = *((int*)pParams);
    while (1) {
        lock(id);

        if (counter % 2) {
            sh[0] = 'H'; sh[1] = 'e'; sh[2] = 'l'; sh[3] = 'l'; sh[4] = 'o'; sh[5]
= '\0';
        } else {
```

```
            sh[0] = 'B'; sh[1] = 'y'; sh[2] = 'e'; sh[3] = '_'; sh[4] = 'u'; sh[5]
= '\0';
        }
        counter++;

        unlock(id);

        usleep(500000);
    }
    return NULL;
}

int main() {
    pthread_t thread_id[2];
    int ids[2] = {0, 1};
    pthread_create(&thread_id[0], NULL, &Thread, &ids[0]);
    pthread_create(&thread_id[1], NULL, &Thread, &ids[1]);

    while (1) {
        printf("%s\n", sh);
        usleep(100000);
    }

    pthread_join(thread_id[0], NULL);
    pthread_join(thread_id[1], NULL);
    return 0;
}
```

peterson.cpp

```
#include <iostream>
#include <atomic>
#include <thread>
#include <chrono>

std::atomic<bool> want[2] = {false, false};
std::atomic<int> turn = 0;

void enter_critical_section(int process_number) {
    int other_process = 1 - process_number;
    want[process_number].store(true, std::memory_order_relaxed);
    turn.store(other_process, std::memory_order_relaxed);
    while (want[other_process].load(std::memory_order_relaxed) &&
turn.load(std::memory_order_relaxed) == other_process) {
        // ждем, пока другой процесс не завершит критическую секцию
    }
    // вход в критическую секцию
    std::cout << "Процесс " << process_number << " вошел в критическую секцию." <<
std::endl;
}
```

```
void exit_critical_section(int process_number) {
    want[process_number].store(false, std::memory_order_relaxed);
    // выход из критической секции
    std::cout << "Процесс " << process_number << " вышел из критической секции." <<
std::endl;
}

void test_algorithm() {
    std::thread t1([&]() {
        enter_critical_section(0);
        // критическая секция для процесса 0
        std::this_thread::sleep_for(std::chrono::seconds(1));
        exit_critical_section(0);
    });

    std::thread t2([&]() {
        enter_critical_section(1);
        // критическая секция для процесса 1
        exit_critical_section(1);
    });

    t1.join();
    t2.join();
}

int main() {
    test_algorithm();
    return 0;
}
```